

Tuesday, 30 December 2025

```
try 'grep --help' for more information.
ubuntu@ip-172-31-7-190:~$ ls
Arti grep
ubuntu@ip-172-31-7-190:~$ cd grep
ubuntu@ip-172-31-7-190:~/grep$ cat test.txt
grep is a Linux command used to search for text patterns inside files or command output.
The name comes from Global Regular Expression Print.
Linux is powerful.
Linux is open source
I like programming
ubuntu@ip-172-31-7-190:~/grep$ grep "source$" test.txt
Linux is open source
ubuntu@ip-172-31-7-190:~/grep$ grep "lin" test.txt
ubuntu@ip-172-31-7-190:~/grep$ grep -i "lin" test.txt
Linux is powerful.
Linux is open source
ubuntu@ip-172-31-7-190:~/grep$ cd ../
ubuntu@ip-172-31-7-190:~$ ls
Arti grep
ubuntu@ip-172-31-7-190:~$ ls
Arti grep
ubuntu@ip-172-31-7-190:~$ pwd
/home/ubuntu
ubuntu@ip-172-31-7-190:~$ mkdir AWK
ubuntu@ip-172-31-7-190:~$ ls
AWK Arti grep
ubuntu@ip-172-31-7-190:~$ cd AWK
ubuntu@ip-172-31-7-190:~/AWK$ cat >> test2.txt
John 25 Engineer
Sara 30 Doctor
Mike 28 Teacher
ubuntu@ip-172-31-7-190:~/AWK$ awk '{print, $1, $3}' test2.txt
awk: cmd. line1: {print, $1, $3}
          ^ syntax error
ubuntu@ip-172-31-7-190:~/AWK$ awk '{print $1, $3}' test2.txt
John Engineer
Sara Doctor
Mike Teacher
ubuntu@ip-172-31-7-190:~/AWK$
```

i-0a311345bceb3e24d (Arti)
PublicIPs: 13.204.66.162 PrivateIPs: 172.31.7.190

CloudShell Feedback Console Mobile App © 2025, Amazon Web Services, Inc. or its affiliates. Privacy Terms Cookie preferences

```
/home/ubuntu
ubuntu@ip-172-31-7-190:~$ mkdir AWK
ubuntu@ip-172-31-7-190:~$ ls
AWK Arti grep
ubuntu@ip-172-31-7-190:~$ cd AWK
ubuntu@ip-172-31-7-190:~/AWK$ cat >> test2.txt
John 25 Engineer
Sara 30 Doctor
Mike 28 Teacher
ubuntu@ip-172-31-7-190:~/AWK$ awk '{print, $1, $3}' test2.txt
awk: cmd. line1: {print, $1, $3}
          ^ syntax error
ubuntu@ip-172-31-7-190:~/AWK$ awk '{print $1, $3}' test2.txt
John Engineer
Sara Doctor
Mike Teacher
ubuntu@ip-172-31-7-190:~/AWK$ awk '{ $2 > 25 { print $1 $2 } }' test2.txt
Sara30
Mike28
ubuntu@ip-172-31-7-190:~/AWK$ awk '{ print NR, $0}' test2.txt
1 John 25 Engineer
2 Sara 30 Doctor
3 Mike 28 Teacher
ubuntu@ip-172-31-7-190:~/AWK$ awk '{print $NF }' test2.txt
Engineer
Doctor
Teacher
ubuntu@ip-172-31-7-190:~/AWK$ awk '{print NF}' test2.txt
3
3
3
ubuntu@ip-172-31-7-190:~/AWK$ awk '{print FS}' test2.txt
ubuntu@ip-172-31-7-190:~/AWK$
```

i-0a311345bceb3e24d (Arti)
PublicIPs: 13.204.66.162 PrivateIPs: 172.31.7.190

CloudShell Feedback Console Mobile App © 2025, Amazon Web Services, Inc. or its affiliates. Privacy Terms Cookie preferences

Grep command

grep stands for **Global Regular Expression Print**.

It is used to **search text or patterns inside files or command output** and print matching lines.

Basic Syntax

```
grep [options] "pattern" filename
```

Basic Example

File **test.txt**:

```
Linux is powerful
Linux is open source
I love DevOps
```

Command:

```
grep "Linux" test.txt
```

Output:

```
Linux is powerful
Linux is open source
```

👉 Finds all lines containing **Linux**

★ Commonly Used **grep** Options (Important for Interviews)

1 **-i** → Ignore case

```
grep -i "linux" test.txt
```

2

Matches Linux, LINUX, linux

2 -n → Show line numbers

```
grep -n "Linux" test.txt
```

Output:

```
1:Linux is powerful
2:Linux is open source
```

3 -w → Match whole word

```
grep -w "Linux" test.txt
```

✓ Matches Linux

✗ Does NOT match LinuxOS

4 -v → Invert match (NOT matching)

```
grep -v "Linux" test.txt
```

Output:

```
I love DevOps
```

5 -c → Count matching lines

```
grep -c "Linux" test.txt
```

Output:

```
2
```

6 -r or -R → Search recursively (directories)

```
grep -r "error" /var/log
```

```
3
```

7 -l → Show only file names with matches

```
grep -l "Linux" *.txt
```

8 -o → Print only matched words

```
grep -o "Linux" test.txt
```

 **Using grep with Pipe (|)**

```
ps -ef | grep root
```

Searches root user processes

 **If Word Does NOT Exist**

```
grep -niw "power" test.txt
```

If word does not exist → **No output shown**
To show a message:

```
grep -niw "power" test.txt || echo "Word not found"
```

 **Advanced Examples (Very Useful)**

Search multiple words

```
grep -E "Linux|DevOps" test.txt
```

Search word at line start

```
grep "^Linux" test.txt
```

Search word at line end

```
grep "DevOps$" test.txt
```

Real DevOps Use Cases

- ✓ Search errors in logs
- ✓ Find running processes
- ✓ Debug application logs
- ✓ Monitor servers

Example:

```
grep -i error /var/log/syslog
```

Quick Summary

Option	Meaning
-i	Ignore case
-n	Line number
-w	Whole word
-v	Exclude match
-c	Count
-r	Recursive
-l	File name only

Exit Status (Important)

Even though nothing is shown, `grep` returns an exit code:

```
echo $?
```

Possible exit codes:

- 0 → match found
- 1 → no match found

- 2 → error (file not found, permission issue, etc.)

How to Show a Message If Word Does NOT Exist

Option 1: Using `||` (most common)

```
grep -niw "power" test.txt || echo "Word not found"
```

Output:

Word not found

Why You See No Message

`grep` is designed for scripting and pipelines, so it:

- Prints matches only
- Uses **exit status** instead of messages

This makes it fast and script-friendly.

AWK command

`awk` is a powerful Linux command used for **text processing and pattern scanning**. It works **line by line**, splits each line into **fields**, and lets you perform **actions** on matching lines.

Basic Syntax

```
awk 'pattern { action }' file
```

- **pattern** → condition to match (optional)
- **action** → what to do when the pattern matches

- If no pattern is given, action runs on **every line**

Simple Example

File: data.txt

```
John 25 Engineer
Sara 30 Doctor
Mike 28 Teacher
```

Print the entire file

```
awk '{ print }' data.txt
```

Fields in awk

By default, fields are separated by **spaces or tabs**.

Symbo l	Meaning
\$0	Entire line
\$1	First field
\$2	Second field
\$3	Third field

Print specific columns

```
awk '{ print $1, $3 }' data.txt
```

Output:

```
John Engineer
Sara Doctor
Mike Teacher
```

Using Conditions (Patterns)

Print lines where age > 25

```
awk '$2 > 25 { print $1, $2 }' data.txt
```

Pattern matching (like grep)

```
awk '/Sara/ { print }' data.txt
```

Field Separator (-F)

File: employees.csv

```
101,John,IT
102,Sara,HR
```

Use comma as delimiter

```
awk -F, '{ print $2, $3 }' employees.csv
```

Built-in Variables

Variable	Meaning
NR	Line number
NF	Number of fields
FS	Field separator
OFS	Output field separator

Print line number and content

```
awk '{ print NR, $0 }' data.txt
```

Print last field

```
awk '{ print $NF }' data.txt
```


BEGIN and END Blocks

BEGIN (before reading file)

```
awk 'BEGIN { print "Name Age Job" } { print }' data.txt
```

END (after reading file)

```
awk 'END { print "Total lines:", NR }' data.txt
```

Using awk with Commands (Pipe)

```
ps -ef | awk '{ print $2, $8 }'
```

(Print PID and command)

Example: Sum a Column

```
awk '{ sum += $2 } END { print sum }' numbers.txt
```